System Programming

X86 Processor Architecture

Dongwann Kang

Associate Professor
Department of Computer Science and Engineering

Basic Language Elements

- 1. First Assembly Language Program
  - *AddTwo* program: a simple assembly language program
    - Adds two numbers
    - Then saves the result in a register

```
1: main PROC
2:    mov eax,5              ; move 5 to the eax register
3:    add eax,6              ; add 6 to the eax register
4:
5:    INVOKE ExitProcess,0   ; end the program
6: main ENDP
```

    - The line numbers are not part of the program

    - [Line 1] starts the main procedure (the entry point for the program)
    - [Line 2] places the integer 5 in the EAX register
    - [Line 3] adds 6 to the value in EAX, giving it a new value of 11
    - [Line 5] calls a Windows service named ExitProcess that halts the program and returns control to the OS
    - [Line 6] the ending marker of the main procedure

Basic Language Elements

- 1. First Assembly Language Program
  - Adding a Variable
    - Saves the results of our addition in a variable named sum

```
 1: .data                       ; this is the data area
 2: sum DWORD 0                 ; create a variable named sum
 3:
 4: .code                       ; this is the code area
 5: main PROC
 6:    mov eax,5                ; move 5 to the eax register
 7:    add eax,6                ; add 6 to the eax register
 8:    mov sum,eax
 9:
10:    INVOKE ExitProcess,0     ; end the program
11: main ENDP
```

      - [Line 2] the sum variable is declared
        - » We give it a size of 32 bits, using the DWORD size keyword
        - » Size keyworks only specify a size
      - The code segment and the data segment
      - Literals, identifiers, directives, instructions, ...

Basic Language Elements

- 2. Integer Literals
  - An integer literal is made up of an optional leading sign, one or more digits, and an optional radix character that indicates the number's base
    - `[{+ | - }] digits [ radix ]`
      - E.g.) 26, 26h, 1101, 1101b
    - The possible radix values

h	hexadecimal	r	encoded real
q/o	octal	t	decimal (alternate)
d	decimal	y	binary (alternate)
b	binary		

  - A hexadecimal literal beginning with a letter must have a leading zero
  - E.g.)

```
26
26d
11010011b
42q
42o
1Ah
0A3h
```

Basic Language Elements

- 3. Constant Integer Expressions
  - A mathematical expression involving integer literals and arithmetic operators
  - Each expression must evaluate to an integer, which can be stored in 32 bits (0 through FFFFFFFFh)
  - They can only be evaluated at assembly time

  - Arithmetic operators

Operator	Name	Precedence Level
( )	Parentheses	1
+, −	Unary plus, minus	2
*, /	Multiply, divide	3
MOD	Modulus	3
+, −	Add, subtract	4

    - Operator precedence: the implied order of operations when an expression contains two or more operators

16 / 5	−3 + 4 * 6 − 1
−(3 + 4) * (6 − 1)	25 mod 3

Basic Language Elements

- 4. Real Number Literals (a.k.a. floating-point literals)
  - Represented as either decimal reals or encoded (hexadecimal) reals
  - Decimal real
    - Contains an optional sign followed by an integer, a decimal point, an optional integer that expresses a fraction, and an optional exponent
      - `[sign]integer.[integer][exponent]`
      - At least one digit and a decimal point are required
      - E.g.)
```
2.
+3.0
-44.2E+05
26.E5
```

```
sign        {+,-}
exponent    E[{+,-}]integer
```

  - Encoded real
    - Represents a real number in hexadecimal, using the IEEE floating-point format for short reals
      - E.g.) The binary representation of decimal +1.0: `0011 1111 1000 0000 0000 0000 0000 0000`
        This value would be encoded as a short real in assembly language as `3F800000r`

Basic Language Elements

- 5. Character Literals
  - A single character enclosed in single or double quotes
  - The assembler stores the value in memory as the character's binary ASCII code
    - E.g.) 'A': the number 65 (or 41 hex) is stored in memory

- 6. String Literals
  - A sequence of characters (including spaces) enclosed in single or double quotes

```
'ABC'                          "This isn't a test"
'X'                            'Say "Good night," Gracie'
"Good night, Gracie"
'4096'
```

  - String literals are stored in memory as sequences of integer byte values
    - E.g.) the string literal "ABCD" contains the four bytes 41h, 42h, 43h, and 44h

Basic Language Elements

- 7. Reserved Words
  - Have special meaning and can only be used in their correct context
  - Not case-sensitive

  - Several types of reserved words
    - Instruction mnemonics, such as MOV, ADD, and MUL
    - Register names
    - Directives, which tell the assembler how to assemble programs
    - Attributes, which provide size and usage information for variables and operands
      - E.g.) BYTE, WORD, ...
    - Operators, used in constant expressions
    - Predefined symbols, such as @data, which return constant integer values at assembly time

Basic Language Elements

- 8. Identifiers
  - A programmer-chosen name
  - It might identify a variable, a constant, a procedure, or a code label

  - Rules on how they can be formed
    - They may contain between 1 and 247 characters
    - They are not case sensitive
    - The first character must be a letter (A..Z, a..z), underscore (_), @ , ?, or $
      - Subsequent characters may also be digits
    - An identifier cannot be the same as an assembler reserved word

```
lineCount     firstValue    index     line_count
myFile        xCoord        main      x_Coord
```

Basic Language Elements

- 9. Directives
  - A command embedded in the source code that is recognized and acted upon by the assembler
  - Directives do not execute at runtime, but they let you define variables, macros, and procedures
  - They can assign names to memory segments and perform many other housekeeping tasks related to the assembler
  - Not case sensitive

  - An example that shows the difference between directives and instructions

```
myVar  DWORD 26
mov    eax,myVar
```

    - The DWORD directive tells the assembler to reserve space in the program for a doubleword variable
    - The MOV instruction executes at runtime, copying the contents of myVar to the EAX register

  - All assemblers for Intel processors share the same instruction set, but they usually have different sets of directives

Basic Language Elements

- 9. Directives
  - Defining Segments
    - One important function of assembler directives is to define program sections, or segments

    - Segments: sections of a program that have different purposes
      - The .DATA directive identifies a segment which can be used to define variables
      - The .CODE directive identifies the area of a program containing executable instructions
      - The .STACK directive identifies the area of a program holding the runtime stack, setting its size:
        - .stack 100h

Basic Language Elements

- 10. Instructions
  - A statement that becomes executable when a program is assembled
  - Translated by the assembler into machine language bytes, which are loaded and executed by the CPU at runtime

  - An instruction contains four basic parts
    - Label (optional)
    - Instruction mnemonic (required)
    - Operand(s) (usually required)
    - Comment (optional)

```
[label:] mnemonic [operands] [;comment]
```

Basic Language Elements

- 10. Instructions
  - Label
    - An identifier that acts as a place marker for instructions and data
    - There are two types of labels: Data labels and Code labels
      - A label placed just before an instruction implies the instruction's address
      - A label placed just before a variable implies the variable's address

    - A *data label* identifies the location of a variable, providing a convenient way to reference the variable in code
      - `count DWORD 100`
        - » Defines a variable named count

      - The assembler assigns a numeric address to each label
      - You can define multiple data items following a label
        - »
```
array DWORD 1024, 2048
      DWORD 4096, 8192
```

Basic Language Elements

- 10. Instructions
  - Label
    - A label in the code area of a program must end with a colon (:) character
    - *Code labels* are used as targets of jumping and looping instructions

    - ```
      target:
          mov   ax,bx
          ...
          jmp   target
      ```
      - JMP instruction transfers control to the location marked by the label named target, creating a loop

    - A code label can share the same line with an instruction, or it can be on a line by itself
      - ```
        L1: mov   ax,bx
        L2:
        ```

    - Label names follow the same rules we described for identifiers
    - You can use the same code label more than once in a program as long as each label is unique within its enclosing procedure

Basic Language Elements

- 10. Instructions
  - Instruction Mnemonic
    - A short word that identifies an instruction
    - Assembly language instruction mnemonics such as mov, add, and sub provide hints about the type of operation they perform

Mnemonic	Description
MOV	Move (assign) one value to another
ADD	Add two values
SUB	Subtract one value from another
MUL	Multiply two values
JMP	Jump to a new location
CALL	Call a procedure

Basic Language Elements

- 10. Instructions
  - Operands
    - A value that is used for input or output for an instruction
    - Assembly language instructions can have between zero and three operands
      - They can be a register, memory operand, integer expression, or input–output port
      - There are different ways to create memory operands: using variable names, registers surrounded by brackets
      - A variable name implies the address of the variable and instructs the computer to reference the contents of memory at the given address

Example	Operand Type
96	*Integer literal*
2 + 4	Integer expression
eax	Register
count	Memory

      - E.g.)

```
stc                     ; set Carry flag
inc  eax                ; add 1 to EAX
mov count,ebx           ; move EBX to count
imul eax,ebx,5
```

      - When instructions have multiple operands, the first one is typically called the *destination operand*
      - The second operand is usually called the *source operand*
      - In general, the contents of the destination operand are modified by the instruction

Basic Language Elements

- 10. Instructions
  - Comments
    - An important way for the writer of a program to communicate information about the program's design to a person reading the source code

    - The following information is typically included at the top of a program listing:
      - Description of the program's purpose
      - Names of persons who created and/or revised the program
      - Program creation and revision dates
      - Technical notes about the program's implementation

    - Single-line comments
      - Beginning with a semicolon character (;)
      - All characters following the semicolon on the same line are ignored by the assembler
    - Block comments
      - Beginning with the COMMENT directive and a user-specified symbol
      - All subsequent lines of text are ignored by the assembler

```
COMMENT  !
     This line is a comment.
     This line is also a comment.
!
```

```
COMMENT &
    This line is a comment.
    This line is also a comment.
&
```

Basic Language Elements

- 10. Instructions
  - The NOP (No Operation) Instruction
    - The safest (and the most useless) instruction is NOP (no operation)
    - It takes up 1 byte of program storage and doesn't do any work
    - It is sometimes used by compilers and assemblers to align code to efficient address boundaries

```
00000000  66 8B C3    mov ax,bx
00000003  90          nop               ; align next instruction
00000004  8B D1       mov edx,ecx
```

      - The first MOV instruction generates three machine code bytes
      - The NOP instruction aligns the address of the third instruction to a doubleword boundary (even multiple of 4)
        - » x86 processors are designed to load code and data more quickly from even doubleword addresses

Example: Adding and Subtracting Integers

- 1. The AddTwo Program

```
 1: ; AddTwo.asm - adds two 32-bit integers
 2: ; Chapter 3 example
 3:
 4: .386
 5: .model flat,stdcall
 6: .stack 4096
 7: ExitProcess PROTO, dwExitCode:DWORD
 8:
 9: .code
10: main PROC
11:    mov    eax,5      ; move 5 to the eax register
12:    add    eax,6      ; add 6 to the eax register
13:
14:    INVOKE ExitProcess,0
15: main ENDP
16: END main
```

  - [Line 4] the .386 directive identifies this as a 32-bit program
  - [Line 5] .model directive selects the program's memory model (*flat*) and identifies the calling convention (named *stdcall*) for procedures
    - 32-bit Windows services require the stdcall convention to be used
  - [Line 6] 4096 bytes of storage for the runtime stack is set

Example: Adding and Subtracting Integers

- 1. The AddTwo Program

```
 1: ; AddTwo.asm - adds two 32-bit integers
 2: ; Chapter 3 example
 3:
 4: .386
 5: .model flat,stdcall
 6: .stack 4096
 7: ExitProcess PROTO, dwExitCode:DWORD
 8:
 9: .code
10: main PROC
11:    mov    eax,5      ; move 5 to the eax register
12:    add    eax,6      ; add 6 to the eax register
13:
14:    INVOKE ExitProcess,0
15: main ENDP
16: END main
```

  - [Line 7] a *prototype* for the ExitProcess function, which is a standard Windows service, is declared
    - A prototype consists of the function name, the PROTO keyword, a comma, and a list of input parameters
      - The input parameter: dwExitCode. Any non-zero return value generally indicates an error code number
      - So, you can think of your assembly programs as subroutines, or processes, which are called by the OS
    - When your program is ready to finish, it calls ExitProcess and returns an integer

Example: Adding and Subtracting Integers

- 1. The AddTwo Program

```
 1: ; AddTwo.asm - adds two 32-bit integers
 2: ; Chapter 3 example
 3:
 4: .386
 5: .model flat,stdcall
 6: .stack 4096
 7: ExitProcess PROTO, dwExitCode:DWORD
 8:
 9: .code
10: main PROC
11:    mov    eax,5      ; move 5 to the eax register
12:    add    eax,6      ; add 6 to the eax register
13:
14:    INVOKE ExitProcess,0
15: main ENDP
16: END main
```

  - [Line 16] the end directive is employed to mark the last line to be assembled, and it identifies the program entry point (main)
    - The label main was declared on [Line 10], and it marks the address at which the program will begin to execute

Example: Adding and Subtracting Integers

- 1. The AddTwo Program
  - A Review of the Assembler Directives
    - The .MODEL directive (`.model flat,stdcall`)
      - Tells the assembler which memory model to use:
        - » In 32-bit programs, we always use the flat memory model
      - The stdcall keyword tells the assembler how to manage the runtime stack when procedures are called
    - The .STACK directive (`.stack 4096`)
      - Tells the assembler how many bytes of memory to reserve for the program's runtime stack
        - » 4096 is probably more than we will ever use, but it happens to correspond to the size of a memory page in the processor's system for managing memory
        - » All modern programs use a stack when calling subroutines
          - First, to hold passed parameters, and second, to hold the address of the code that called the function
          - The CPU uses this address to return when the function call finishes, back to the spot where the function was called
        - » In addition, the runtime stack can hold local variables

Example: Adding and Subtracting Integers

- 1. The AddTwo Program
  - A Review of the Assembler Directives
    - The .CODE directive (`.code`)
      - Marks the beginning of the code area of a program, the area that contains executable instructions
      - Usually the next line after .CODE is the declaration of the program's entry point, and by convention, it is usually a procedure named main
      - The entry point of a program is the location of the very first instruction the program will execute

```
.code
main PROC
```

    - The ENDP directive (`main ENDP`)
      - Marks the end of a procedure
      - Our program had a procedure named main, so the endp must use the same name
    - The END directive (`END main`)
      - Marks the end of the program, and references the program entry point
      - If you add any more lines to a program after the END directive, they will be ignored by the assembler

Example: Adding and Subtracting Integers

- 2. Running and Debugging the AddTwo Program
  - How to create the AddTwo program
    - Creating a MASM project
      1. Run Visual Studio 2019
      2. [Create a new project]
      3. Select [Empty Project]
      4. Enter the name of project and select [Create]
      5. Right-click the project name in [Solution Explorer], select [Add] – [New Item]
      6. Select [C++ File (.cpp)], enter file name with .asm extension (e.g. source.asm), and click [Add]

    - Project Setting
      1. Right-click the project name in [Solution Explorer], select [Build Dependencies] – [Build Customizations], and check [masm] option
      2. Right-click the source file (*.asm) in [Solution Explorer], select [Properties] – [General] – [Item Type] – [Microsoft Macro Assembler]
      3. Right-click the project name in [Solution Explorer], select [Properties] – [Linker] – [Advanced] – [Image Has Safe Exception Handlers] – [No]

Example: Adding and Subtracting Integers

- 2. Running and Debugging the AddTwo Program
  - Customizing the Debugging Interface
    1. While the program is being debugged, select [Debug] – [Windows] – [Registers]
    2. Right-click the [Registers] window and select [Flags]

  - Syntax highlighting
    1. Download the Usertype.dat file from the e-class
    2. Close Visual Studio, if it is running
    3. Copy Usertype.dat to the below folder
       - C:\Program Files\Microsoft Visual Studio\2022\Community \Common7\IDE
    4. Open Visual Studio, select [Options] from the [Tools] menu, select [Text Editor], and select [File Extension]
    5. Enter [asm] as the extension, select [Microsoft Visual C++] from the [Editor list], and click the [Add] – [OK]
    6. Open your project

Example: Adding and Subtracting Integers

- 3. Program Template
  - Assembly language programs have a simple structure, with small variations
  - When you begin a new program, it helps to start with an empty shell program with all basic elements in place

```
; Program template (Template.asm)

.386
.model flat,stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:DWORD

.data
     ; declare variables here
.code
main PROC
     ; write your code here

     INVOKE ExitProcess,0
main ENDP
END main
```

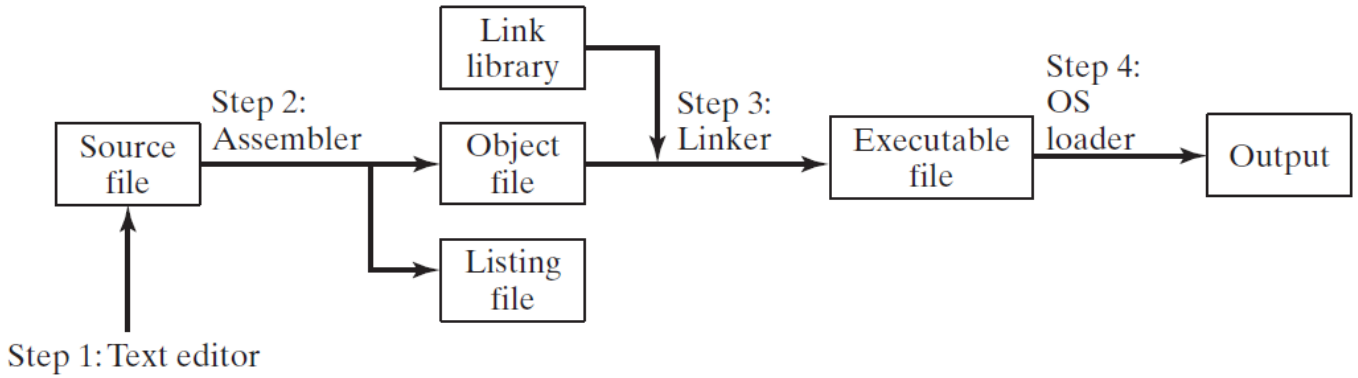
Assembling, Linking, and Running Programs

- A source program written in assembly language cannot be executed directly
  - It must be translated, or assembled into executable code
  - An assembler is very similar to a compiler

- The assembler produces a file containing machine language called an *object file*
  - Object file isn't quite ready to execute
  - It must be passed to another program called a *linker*, which in turn produces an *executable file*

Assembling, Linking, and Running Programs

- 1. The Assemble-Link-Execute Cycle
  - [Step 1] A programmer uses a text editor to create an ASCII text file named the source file
  - [Step 2] The assembler reads the source file and produces an object file, a machine-language translation of the program. Optionally, it produces a listing file
    - If any errors occur, the programmer must return to [Step 1] and fix the program
  - [Step 3] The linker reads the object file and checks to see if the program contains any calls to procedures in a link library. The linker copies any required procedures from the link library, combines them with the object file, and produces the executable file
  - [Step 4] The OS loader utility reads the executable file into memory and branches the CPU to the program's starting address, and the program begins to execute.



Assembling, Linking, and Running Programs

- 2. Listing File
  - A listing file contains the following:
    - A copy of the program's source code, with line numbers, the numeric address of each instruction, the machine code bytes of each instruction (in hexadecimal), and a symbol table
    - The symbol table contains the names of all program identifiers, segments, and related information

```
 1:    ; AddTwo.asm - adds two 32-bit integers.
 2:    ; Chapter 3 example
 3:
 4:    .386
 5:    .model flat,stdcall
 6:    .stack 4096
 7:    ExitProcess PROTO,dwExitCode:DWORD
 8:
 9:     00000000                        .code
10:     00000000                       main PROC
11:     00000000  B8 00000005             mov  eax,5
12:     00000005  83 C0 06                add  eax,6
13:
14:                                       invoke ExitProcess,0
15:     00000008  6A 00                   push   +000000000h
16:     0000000A  E8 00000000 E           call   ExitProcess
17:     0000000F                       main ENDP
18:                                    END main
```

Assembling, Linking, and Running Programs

- 2. Listing File
  - A sample listing file
    - [Lines 1–7] no executable code
    - [Line 9] the beginning of the code segment is located at address 00000000h
      - This address is relative to the beginning of the program's memory footprint, but it will be converted into an absolute memory address when the program is loaded into memory
    - [Lines 10–11] the same starting address of 00000000
    - [Lines 11] the first executable statement
      - Hexadecimal bytes B8 00000005 represent the machine code instruction (B8), and the constant 32-bit value (00000005) that is assigned to EAX by the instruction
      - The value B8: an operation code (or just opcode)

```
 1:    ; AddTwo.asm - adds two 32-bit integers.
 2:    ; Chapter 3 example
 3:
 4:    .386
 5:    .model flat,stdcall
 6:    .stack 4096
 7:    ExitProcess PROTO,dwExitCode:DWORD
 8:
 9:     00000000                       .code
10:     00000000                       main PROC
11:     00000000  B8 00000005             mov  eax,5
12:     00000005  83 C0 06                add  eax,6
13:
14:                                       invoke ExitProcess,0
15:     00000008  6A 00                   push   +000000000h
16:     0000000A  E8 00000000 E           call   ExitProcess
17:     0000000F                       main ENDP
18:                                    END main
```

Assembling, Linking, and Running Programs

- 2. Listing File
  - A sample listing file
    - [Line 12] an executable instruction, starting at offset 00000005
      - That offset is a distance of 5 bytes from the beginning of the program
    - [Line 14] the invoke directive
      - Lines 15 and 16 seem to have been inserted into the code
      - The INVOKE directive causes the assembler to generate the PUSH and CALL statements

    - The machine instructions are loaded into memory as a sequence of integer values
      - B8, 00000005, 83, C0, 06, 6A, 00, EB, 00000000
      - These machine instructions are exactly 15 bytes long

```
 1:    ; AddTwo.asm - adds two 32-bit integers.
 2:    ; Chapter 3 example
 3:
 4:    .386
 5:    .model flat,stdcall
 6:    .stack 4096
 7:    ExitProcess PROTO,dwExitCode:DWORD
 8:
 9:     00000000                       .code
10:     00000000                       main PROC
11:     00000000  B8 00000005             mov  eax,5
12:     00000005  83 C0 06                add  eax,6
13:
14:                                       invoke ExitProcess,0
15:     00000008  6A 00                   push   +000000000h
16:     0000000A  E8 00000000 E           call   ExitProcess
17:     0000000F                       main ENDP
18:                                    END main
```

Assembling, Linking, and Running Programs

- 2. Listing File
  - How to configure Visual Studio to generate a listing file
    - Open your project
    - Select [Properties] from the [Project] menu
    - Under [Configuration Properties], select [Microsoft Macro Assembler]
    - Then select [Listing File]
    - Set [Generate Preprocessed Source Listing] to [Yes]
    - Set [List All Available Information] to [Yes]
    - Set [Assembled Code Listing File] to [$(ProjectName).lst]

Defining Data

- 1. Intrinsic Data Types
  - Describe types in terms of their size (byte, word, doubleword, and so on), whether they are signed, and whether they are integers or reals
  - There's a fair amount of overlap in these types
    - E.g.) the DWORD type is interchangeable with the SDWORD type
  - The assembler only evaluates the sizes of operands
    - E.g.) you can only assign variables of type DWORD, SDWORD, or REAL4 to a 32-bit integer

Type	Usage
BYTE	8-bit unsigned integer. B stands for byte
SBYTE	8-bit signed integer. S stands for signed
WORD	16-bit unsigned integer
SWORD	16-bit signed integer
DWORD	32-bit unsigned integer. D stands for double
SDWORD	32-bit signed integer. SD stands for signed double
FWORD	48-bit integer (Far pointer in protected mode)
QWORD	64-bit integer. Q stands for quad
TBYTE	80-bit (10-byte) integer. T stands for Ten-byte
REAL4	32-bit (4-byte) IEEE short real
REAL8	64-bit (8-byte) IEEE long real
REAL10	80-bit (10-byte) IEEE extended real

Defining Data

- 2. Data Definition Statement
  - Sets aside storage in memory for a variable, with an optional name
  - Creates variables based on intrinsic data types
  - The syntax: `[name] directive initializer [,initializer]...`
    - An example: `count DWORD 12345`
    - Name
      - The optional name assigned to a variable must conform to the rules for identifiers
    - Directive
      - The directive in a data definition statement can be BYTE, WORD, DWORD, SBYTE, SWORD, or any of the types
      - In addition, it can be any of the legacy data definition directives
    - Initializer
      - At least one initializer is required in a data definition, even if it is zero
      - Additional initializers, if any, are separated by commas
      - For integer data types, initializer is an integer literal or integer expression matching the size of the variable's type, such as BYTE or WORD
      - If you prefer to leave the variable uninitialized, the ? symbol can be used as the initializer
      - All initializers, regardless of their format, are converted to binary data by the assembler

Legacy data directives

Directive	Usage
DB	8-bit integer
DW	16-bit integer
DD	32-bit integer or real
DQ	64-bit integer or real
DT	define 80-bit (10-byte) integer

Defining Data

- 3. Adding a Variable to the AddTwo Program
  - Run this in the debugger by setting a breakpoint on [line 13] and stepping through the program one line at a time
  - Check the value of the sum variable after executing [line 15]

```
 1: ; AddTwoSum.asm - Chapter 3 example
 2:
 3:    .386
 4:    .model flat,stdcall
 5:    .stack 4096
 6:    ExitProcess PROTO, dwExitCode:DWORD
 7:
 8:    .data
 9:    sum DWORD 0
10:
11:    .code
12:    main PROC
13:       mov eax,5
14:       add eax,6
15:       mov sum,eax
16:
17:       INVOKE ExitProcess,0
18:    main ENDP
19:    END main
```

Defining Data

- 4. Defining BYTE and SBYTE Data
  - Allocate storage for one or more unsigned or signed values
  - Each initializer must fit into 8 bits of storage

```
value1 BYTE   'A'              ; character literal
value2 BYTE    0               ; smallest unsigned byte
value3 BYTE  255               ; largest unsigned byte
value4 SBYTE -128              ; smallest signed byte
value5 SBYTE +127              ; largest signed byte
value6 BYTE ?
```

  - The optional name is a label marking the variable's offset from the beginning of its enclosing segment
    - If value1 is located at offset 0000 in the data segment and consumes one byte of storage, value2 is automatically located at offset 0001

  - The DB directive can also define an 8-bit variable, signed or unsigned

```
val1 DB 255                    ; unsigned byte
val2 DB -128                   ; signed byte
```

Defining Data

- 4. Defining BYTE and SBYTE Data
  - Multiple Initializers
    - If multiple initializers are used, its label refers only to the offset of the first initializer

```
list BYTE 10,20,30,40
```

Offset	Value
0000:	10
0001:	20
0002:	30
0003:	40

    - Not all data definitions require labels
      - To continue the array of bytes begun with any label, we can define additional bytes on the next lines:

```
list BYTE 10,20,30,40
     BYTE 50,60,70,80
     BYTE 81,82,83,84
```

    - Within a single data definition, its initializers can use different radixes

```
list1 BYTE 10, 32, 41h, 00100010b
list2 BYTE 0Ah, 20h, 'A', 22h
```

Defining Data

- 4. Defining BYTE and SBYTE Data
  - Defining Strings
    - To define a string of characters, enclose them in single or double quotation marks

    - A null-terminated string

```
greeting1 BYTE "Good afternoon",0
greeting2 BYTE 'Good night',0
```

      - The most common type of string ends with a null byte (containing 0)
      - Used in many programming languages
      - A string can be divided between multiple lines without having to supply a label for each line

```
greeting1 BYTE "Welcome to the Encryption Demo program "
  BYTE "created by Kip Irvine.",0dh,0ah
  BYTE "If you wish to modify this program, please "
  BYTE "send me a copy.",0dh,0ah,0
```

      - The hexadecimal codes 0Dh and 0Ah are alternately called CR/LF or end-of-line characters
        - When written to standard output, they move the cursor to the left column of the line following the current line
      - The line continuation character (\) concatenates two source code lines into a single statement
        - It must be the last character on the line

```
greeting1 \
BYTE "Welcome to the Encryption Demo program "
```

Defining Data

- 4. Defining BYTE and SBYTE Data
  - DUP Operator
    - Allocates storage for multiple data items, using an integer expression as a counter
    - Particularly useful when allocating space for a string or array
    - can be used with initialized or uninitialized data

```
BYTE 20 DUP(0)                  ; 20 bytes, all equal to zero
BYTE 20 DUP(?)                  ; 20 bytes, uninitialized
BYTE  4 DUP("STACK")            ; 20 bytes: "STACKSTACKSTACKSTACK"
```

Defining Data

- 5. Defining WORD and SWORD Data
  - Create storage for one or more 16-bit integers

```
word1   WORD    65535           ; largest unsigned value
word2   SWORD   -32768          ; smallest signed value
word3   WORD    ?               ; uninitialized, unsigned

val1    DW 65535                ; unsigned
val2    DW -32768               ; signed
```

  - Array of 16-Bit Words

```
array WORD 5 DUP(?)             ; 5 values, uninitialized
```

Offset	Value
0000:	1
0002:	2
0004:	3
0006:	4
0008:	5

Defining Data

- 6. Defining DWORD and SDWORD Data
  - Allocate storage for one or more 32-bit integers

```
val1 DWORD   12345678h          ; unsigned
val2 SDWORD -2147483648         ; signed
val3 DWORD   20 DUP(?)          ; unsigned array

val1 DD 12345678h               ; unsigned
val2 DD -2147483648             ; signed
```

  - The DWORD can be used to declare a variable that contains the 32-bit offset of another variable
    - Below, pVal contains the offset of val3

```
pVal DWORD val3
```

  - Array of 32-Bt Doublewords

```
myList DWORD 1,2,3,4,5
```

Offset	Value
0000:	1
0004:	2
0008:	3
000C:	4
0010:	5

Defining Data

- 7. Defining QWORD Data
  - Allocates storage for 64-bit (8-byte) values

```
quad1 QWORD 1234567812345678h
quad1 DQ 1234567812345678h
```

- 8. Defining Floating-Point Types
  - REAL4 defines a 4-byte single-precision floating-point variable
  - REAL8 defines an 8-byte double-precision value
  - REAL10 defines a 10-byte extended-precision value
  - Each requires one or more real constant initializers

```
rVal1      REAL4 -1.2
rVal2      REAL8  3.2E-260
rVal3      REAL10 4.6E+4096
ShortArray REAL4  20 DUP(0.0)
```

    - The DD, DQ, and DT directives can define also real numbers

```
rVal1 DD -1.2                  ; short real
rVal2 DQ  3.2E-260             ; long real
rVal3 DT  4.6E+4096            ; extended-precision real
```

Defining Data

- 9. A Program That Adds Variables

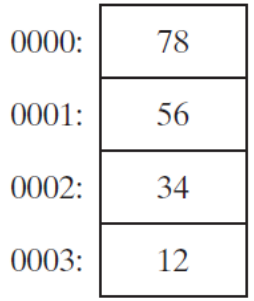
```
 1: ; AddVariables.asm - Chapter 3 example
 2:
 3:    .386
 4:    .model flat,stdcall
 5:    .stack 4096
 6:    ExitProcess PROTO, dwExitCode:DWORD
 7:
 8:    .data
 9:    firstval  DWORD 20002000h
10:    secondval DWORD 11111111h
11:    thirdval  DWORD 22222222h
12:    sum       DWORD 0
13:
14:    .code
15:    main PROC
16:       mov eax,firstval
17:       add eax,secondval
18:       add eax,thirdval
19:       mov sum,eax
20:
21:       INVOKE ExitProcess,0
22:    main ENDP
23:    END main
```

  - [Lines 9–11] initialized three variables with nonzero values
  - [Lines 16–18] add the variables
    - The x86 instruction set does not let us add one variable directly to another, but it does allow a variable to be added to a register (That is why [lines 16–17] use EAX)
  - [Line 19] the sum is copied into the variable named sum

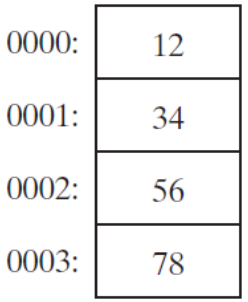
Defining Data

- 10. Little-Endian Order
  - x86 processors store and retrieve data from memory using little-endian order (low to high)
  - The LSB is stored at the first memory address allocated for the data
  - The remaining bytes are stored in the next consecutive memory positions

  - E.g) the doubleword 12345678h



  - Some other computer systems use big-endian order (high to low)



Defining Data

- 11. Declaring Uninitialized Data
  - The .DATA? directive declares uninitialized data
  - When defining a large block of uninitialized data, the .DATA? directive reduces the size of a compiled program

```
.data
smallArray DWORD 10 DUP(0)      ; 40 bytes
.data?
bigArray DWORD 5000 DUP(?)      ; 20,000 bytes, not initialized
```

    - On the other hand, the following code produces a compiled program 20,000 bytes larger

```
.data
smallArray DWORD 10 DUP(0)      ; 40 bytes
bigArray DWORD 5000 DUP(?)      ; 20,000 bytes
```

  - Mixing Code and Data
    - The assembler lets you switch back and forth between code and data in your programs

```
.code
mov eax,ebx
.data
temp DWORD ?
.code
mov temp,eax
```

Symbolic Constants

- A symbolic constant is created by associating an identifier (a symbol) with an integer expression or some text
- Symbols do not reserve storage
  - They are used only by the assembler when scanning a program
  - They cannot change at runtime

	Symbol	Variable
Uses storage?	No	Yes
Value changes at runtime?	No	Yes

Symbolic Constants

- 1. Equal-Sign Directive
  - Associates a symbol name with an integer expression
  - Syntax: `name = expression`
    - Ordinarily, expression is a 32-bit integer value
    - When a program is assembled, all occurrences of *name* are replaced by *expression* during the assembler's preprocessor step

    - E.g.)
      ```
      COUNT = 500
            ⋮
      mov eax, COUNT
      ```

      - When the file is assembled, MASM will scan the source file and produce the corresponding code lines:

      ```
      mov eax, 500
      ```

  - Why Use Symbols?
    - Programs are easier to read and maintain if symbols are used

Symbolic Constants

- 1. Equal-Sign Directive
  - Current Location Counter ($)
    - One of the most important symbols of all
    - For example, the following declaration declares a variable named selfPtr and initializes it with the variable's offset value:

```
selfPtr DWORD $
```

  - Keyboard Definitions
    - Programs often define symbols that identify commonly used numeric keyboard codes

```
Esc_key = 27
mov  al,Esc_key                ; good style
mov  al,27                     ; poor style
```

  - Using the DUP Operator
    - The counter used by DUP should be a symbolic constant, to simplify program maintenance

```
array dword COUNT DUP(0)
```

  - Redefinitions
    - A symbol defined with = can be redefined within the same program
    - The changing value of a symbol has nothing to do with the runtime execution order of statements

```
COUNT = 5
mov al,COUNT       ; AL = 5
COUNT = 10
mov al,COUNT       ; AL = 10
COUNT = 100
mov al,COUNT       ; AL = 100
```

Symbolic Constants

- 2. Calculating the Sizes of Arrays and Strings
  - Symbolic constant for array size

```
list BYTE 10,20,30,40
ListSize = 4
```

    - Explicitly stating an array's size can lead to a programming error, particularly if you should later insert or remove array elements

  - A better way: to let the assembler calculate its value for you
    - The $ operator (current location counter) returns the offset associated with the current program statement

```
list BYTE 10,20,30,40
ListSize = ($ - list)
```

      - ListSize must follow immediately after list

```
myString  BYTE "This is a long string, containing"
          BYTE "any number of characters"
myString_len = ($ - myString)
```

Symbolic Constants

- 2. Calculating the Sizes of Arrays and Strings
  - Arrays of Words and DoubleWords
    - When calculating the number of elements in an array containing values other than bytes, you should always divide the total array size (in bytes) by the size of the individual array elements

```
list  WORD  1000h,2000h,3000h,4000h
ListSize = ($ - list) / 2

list  DWORD  10000000h,20000000h,30000000h,40000000h
ListSize = ($ - list) / 4
```

Symbolic Constants

- 3. EQU Directive
  - Associates a symbolic name with an integer expression or some arbitrary text
  - There are three formats:

```
name EQU expression
name EQU symbol
name EQU <text>
```

    - In the first format, *expression* must be a valid integer expression
    - In the second format, *symbol* is an existing symbol name, already defined with = or EQU
    - In the third format, any text may appear within the brackets <. . .>
  - When the assembler encounters name later in the program, it substitutes the integer value or text for the symbol
  - EQU can be useful when defining a value that does not evaluate to an integer

```
PI EQU <3.1416>
pressKey EQU <"Press any key to continue...",0>
.
.
.data
prompt  BYTE   pressKey
```

Symbolic Constants

- 3. EQU Directive
  - An example

```
matrix1  EQU   10 * 10
matrix2  EQU  <10 * 10>
.data
M1 WORD matrix1
M2 WORD matrix2
```

    - The assembler produces different data definitions for M1 and M2
    - The integer expression in matrix1 is evaluated and assigned to M1
    - On the other hand, the text in matrix2 is copied directly into the data definition for M2

```
M1 WORD  100
M2 WORD  10 * 10
```

  - No Redefinition
    - Unlike the = directive, a symbol defined with EQU cannot be redefined in the same source code file
    - This restriction prevents an existing symbol from being inadvertently assigned a new value

Symbolic Constants

- 4. TEXTEQU Directive
  - Creates what is known as a text macro
  - There are three different formats:

```
name TEXTEQU <text>
name TEXTEQU textmacro
name TEXTEQU %constExpr
```

    - The first assigns text
    - The second assigns the contents of an existing text macro
    - The third assigns a constant integer expression

```
continueMsg TEXTEQU <"Do you wish to continue (Y/N)?">
.data
prompt1 BYTE continueMsg
```

    - Text macros can build on each other

```
rowSize = 5
count   TEXTEQU  %(rowSize * 2)
move    TEXTEQU  <mov>
setupAL TEXTEQU  <move al,count>
```

setupAL  mov al,10

    - A symbol defined by TEXTEQU can be redefined at any time